

Fundamentals of Software Testing

Strategies and Techniques

Motivation: Why Do We Test?

- **Spectacular Failures:**

Software bugs have led to catastrophic losses, such as the **Mariner 1** spacecraft failure (1962) due to a transcription error and the **Ariane 5** rocket self-destruction (1996) costing \$1 billion.

- **Economic Impact:**

A NIST report estimated that inadequate testing infrastructure costs the US economy \$22–\$59 billion annually.

- **Core Goals:**

To uncover as many errors as possible, demonstrate requirements fulfillment, and validate software quality.

- Testing and Debugging are fundamental processes in software engineering.
- While they often appear similar to the uninitiated, they are quite distinct in their goals and methods.
- Understanding these differences is crucial for effective software development.

Understanding Software Testing

- What is Testing?
 - It is the process of verifying and validating that a software application is bug-free.
 - It ensures the software meets technical requirements as guided by design and development.
- Purpose of Software Testing
 - To identify errors, faults, or missing requirements.
 - To compare the actual software performance against specified requirements.
- Meeting User Requirements
 - Testing ensures user requirements are met effectively and efficiently.
 - It involves checking how the system handles related scenarios, including exceptional and boundary cases.

Fundamental Definitions

- **Error:** A mistake made by a programmer (e.g., misunderstanding requirements).
- **Fault/Bug/Defect:** The manifestation of an error within the code (e.g., using a '-' sign instead of '+').
- **Failure:** The observed deviation of program behaviour from expected results.
- **Testing vs. Debugging:**
 - **Testing:** Process of identifying defects or errors in the software system.
 - **Debugging:** Process to fix those identified defects or errors that affect software operation.
- **Verification vs. Validation:** Verification asks "Are we building the product right?"; Validation asks "Are we building the right product?".

Types of Software Testing

Testing is categorized into several distinct methodologies:

- Manual Testing - It often involves:
 - **White Box Testing:** Focusing on internal structures.
 - **Black Box Testing:** Focusing on functional requirements.
 - **Gray Box Testing:** A combination of both.
- Automated Testing
 - Involves the use of software tools to execute test cases.
 - Often used for repetitive but necessary tasks to improve efficiency.
- Functional Testing
- Non-Functional Testing - Focuses on the "how" the system works rather than "what" it does:
 - **Performance Testing**
 - **Usability Testing**
 - **Compatibility Testing** ...

Levels of Testing (Functional)

Software products are generally tested at three primary levels:

- 1 **Unit Testing:** Testing individual modules in isolation to reduce debugging effort.
- 2 **Integration Testing:** Testing modules as they are combined to ensure they work together correctly.
- 3 **System Testing:** Validating a fully developed system against its requirements (SRS).

Integration Testing Strategies

- **Big Bang:** All modules are integrated at once. Difficult to localize errors.
- **Bottom-up:** Bottom-level modules are tested first. Good for fault isolation but major design faults may appear late.
- **Top-down:** Starts with the main routine using 'stubs.' Major design faults show up early.
- **Mixed (Sandwich):** Combines top-down and bottom-up approaches.

Integration Testing Strategies ... cont'd

- Types: Incremental or Phased.
 - In incremental integration testing, only one new module is added to the partial system each time.
 - In phased integration, a group of related modules are added to the partially integrated system each time.
- Big-bang testing: A degenerate case of phased integration testing.
- Remember
 - Phased integration requires less number of integration steps compared to the incremental integration approach.
 - However, when failures are detected, it is easier to debug if using incremental testing since errors are very likely to be in the newly integrated module.

System Testing Strategies

- Three main kinds:

- 1 **Alpha Testing** - System testing is carried out by the test team within the developing organisation.
- 2 **Beta Testing** - System testing performed by a select group of friendly customers.
- 3 **Acceptance Testing** - System testing performed by the customer himself to determine whether the system should be accepted or rejected.

How?

- Test software with the client's environment and real data
 - Performed by SQA team in the presence of client representatives, or an independent SQA team (designated third party) hired by the client.
- During system testing:
 - Functional requirements are validated through functional tests.
 - Non-functional requirements validated through performance tests.

Testing Activities and its Elements

- **Key Activities**

- Test Suite Design
- Run test cases and observe results to detect failures.
- Debug to locate errors
- Correct errors.

- **Key Elements** - Test cases and Test suites

- Test software using a carefully designed set of test cases.
- The set of all test cases is called the Test Suite.
- A test case is a triplet [I,S,O]
 - I is the data to be input to the system,
 - S is the state of the system at which the data will be input,
 - O is the expected output of the system.

Design of Test Cases

- Exhaustive testing of any non-trivial system is impractical as input data domain is extremely large.
- Design an optimal test suite of reasonable size, that Uncovers as many errors as possible.
- If test cases are selected randomly:
 - Many test cases would not contribute to the significance of test suite,
 - Would not detect errors not already being detected by other test cases in the suite.
- Number of test cases in a randomly selected test suite is **NOT** an indication of effectiveness of testing.

Design of Test Cases ...cont'd

- Testing a system using a large number of randomly selected test cases does not mean that many errors in the system will be uncovered.
- Consider an example to find the maximum of two integers x and y . Following code was considered:

```
If (x>y) max = x;  
else max = x; // Ahh! simple programming error
```

- Test Suite
 - Small (2 TCs) test suite ($x=3,y=2$);($x=2,y=3$) can detect the error.
 - Larger (3 TCs) test suite ($x=3,y=2$);($x=4,y=3$); ($x=5,y=1$) does not detect the error.
- Systematic approaches are required to design an optimal test suite.
- Each test case in the suite should detect different errors.

Black Box Testing (Functional)

- **Definition:** Testing based solely on functional specifications without knowledge of internal code structure.
- **Equivalence Class Partitioning (ECP):**
 - Divide input domain into classes where the program is expected to behave similarly.
 - Identify one valid class and two invalid classes (e.g., if range is 1–5000, classes are < 1 , $1 - 5000$, and > 5000).
- **Boundary Value Analysis (BVA):**
 - Errors often occur at the edges of partitions.
 - Test values *on* and *around* the boundaries (e.g., for range 1–5000, test $\{0, 1, 5000, 5001\}$).

Black Box Testing

Equivalence Class Partitioning (ECP) & Boundary Value Analysis (BVA)

Understanding Black Box Testing

- Testing technique focusing on input-output behavior
- No knowledge of internal code structure required
- Used in:
 - Functional Testing
 - System Testing
 - Acceptance Testing
- Two important techniques:
 - Equivalence Class Partitioning (ECP)
 - Boundary Value Analysis (BVA)

Equivalence Class Partitioning (ECP)

- Divides input data into logical partitions
- Each partition behaves similarly
- Only one representative value per class is tested
- Reduces the number of test cases significantly

C++ Program: Grade Classification

```
#include <iostream>
using namespace std;

string getGrade(int marks) {
    if (marks < 0 || marks > 100) return "Invalid";
    else if (marks < 40) return "Fail";
    else if (marks < 60) return "Pass";
    else if (marks < 75) return "First-Class";
    else return "Distinction";
}

int main() {
    cout << getGrade(65);
}
```

Equivalence Classes

- Invalid Inputs:
 - marks < 0
 - marks > 100
- Valid Inputs:
 - 0–39 $\rightarrow$ Fail
 - 40–59 $\rightarrow$ Pass
 - 60–74 $\rightarrow$ First Class
 - 75–100 $\rightarrow$ Distinction

ECP Test Cases

Test Case	Input	Expected Output
TC1	-10	Invalid
TC2	20	Fail
TC3	50	Pass
TC4	70	First Class
TC5	90	Distinction
TC6	110	Invalid

Do you know?

If ECP was not used, how many test cases were needed?

Boundary Value Analysis (BVA)

- Focuses on edge values of input domain
- Errors are more likely at boundaries
- Tests:
 - Just below boundary
 - At boundary
 - Just above boundary

C++ Program: Age Eligibility

```
#include <iostream>
using namespace std;

string checkEligibility(int age) {
    if (age < 18) return "Not-Eligible";
    else if (age <= 60) return "Eligible";
    else return "Senior-Citizen";
}

int main() {
    cout << checkEligibility(18);
}
```

Identified Boundaries

- Lower Boundary: 18
- Upper Boundary: 60

BVA Test Cases

Test Case	Input	Expected Output
TC1	17	Not Eligible
TC2	18	Eligible
TC3	19	Eligible
TC4	59	Eligible
TC5	60	Eligible
TC6	61	Senior Citizen

Comparison: ECP vs BVA

Aspect	ECP	BVA
Focus	Input classes	Boundary values
Test Cases	Fewer	More
Goal	Coverage	Edge defect detection

Remember

- Black-box testing validates functionality from user perspective
- ECP reduces redundancy in testing
- BVA improves defect detection at critical edges
- Both techniques should be used together for effective testing